

Planet Blitz — AI 활용 기술 문서

NAN 2026 Game×AI 해커톤 사전 과제 · 개인 참가 김민석 (v0o0v2@gmail.com) · 2026-08-10

0. 요약 — 무엇을 만들었고, 무엇을 지휘했는가

Planet Blitz는 1인이 26일 동안 만든 온라인 게임이고, 게임 코드와 테스트 중 사람이 직접 타이핑한 줄은 없습니다. 전부 Claude Code가 작성했습니다. 제가 맡은 일은 AI를 지휘하는 체계를 설계하고 운영하는 쪽이었습니다. 요구를 확정하는 인터뷰, 결정을 붙잡아 두는 ADR, 작성자와 비평자를 갈라 놓은 멀티에이전트 파이프라인, AI 산출물을 기계적으로 걸러 내는 검증 게이트가 그 체계를 이룹니다.

그 체계가 낸 결과의 규모입니다(2026-07-15 ~ 08-09 실측, public 리포에서 직접 확인 가능).

지표	값
커밋 / 머지된 PR	1,392 커밋 / 498 PR
TypeScript	872파일 · 약 29만 줄
자동 테스트	378파일 · 8,676건 전량 통과 (약 82초)
아키텍처 결정 기록(ADR)	55건
DB 마이그레이션 / Edge Function	63건 / 3종 (서버 권위 쓰기)
픽셀아트 에셋	PNG 573장 (AI 생성)
레인 계획·조사·인수인계 문서	161건 (.omc/ 아래)

전량 AI 작성이라는 말은 직접 세어서 확인하실 수 있습니다. 총 1,392 커밋은 비머지 894건과 머지 498건으로 나뉘는데, 비머지 894건 가운데 849건(95%)이 Co-Authored-By: Claude trailer를 담니다. trailer가 없는 45건도 같은 AI 워크플로의 산출이고, 일부 세션 설정에서 trailer가 누락됐을 뿐입니다. 아래 명령으로 세어 보실 수 있습니다(2026-08-09 기준).

```
git rev-list --count --no-merges HEAD # 894
git log --no-merges --grep="Co-Authored-By: Claude" --format=%H | wc -l # 849
```

1. 아키텍처 — AI가 감당할 수 있게 설계했다

AI에게 대량의 코드를 맡길 때 가장 큰 위험은 그럴듯하지만 틀린 코드입니다. 이 프로젝트는 아키텍처 자체를 기계가 검증할 수 있는 형태로 설계해 그 위험을 구조적으로 낮췄습니다.

- 결정론 시뮬 코어**(src/sim)는 순수 TypeScript로, 시드 RNG만 씁니다. 렌더·시간·난수 API를 쓰면 ESLint 에러가 납니다. 지켜 달라고 부탁하는 대신 빌드가 막게 했습니다.
- 렌더 계층**(PixiJS v8)은 시뮬이 뱉은 스냅샷만 소비하고 거꾸로 흘러들지 못합니다. 화면 버그가 게임 로직을 오염시킬 수 없습니다.
- 서버**(Supabase Postgres + Edge Functions)는 보상 발급과 래더 변동 같은 이득 쓰기를 service_role로만 처리합니다. AI가 짠 클라이언트를 신뢰하지 않는다는 전제 위에 온라인 게임을 올렸습니다.

결정론은 그대로 테스트 가능성이 됩니다. 같은 시드에서 같은 결과가 보장되므로, AI가 고친 코드가 게임 결과를 바꿨는지를 틱 단위 상태 해시 비교로 기계가 판정합니다. 사람이 눈으로 확인하지 않아도 되니 AI의 대량 산출을 받아 낼 수 있습니다.

서버가 리플레이를 전수 재실행해 검증하던 설계는 의도적으로 걷어냈습니다(ADR-0050). 재실행은 서버가 시물 전체를 싣게 만들어 배포 결합을 키우는 데 비해 막아 주는 폭이 좁았습니다. 대신 보상은 서버가 자기 시드로 직접 굴리고, 획득 원장과 개연성 캡으로 위조 이득을 정직한 최상위 플레이어 수준 안에 가두는 쪽으로 바꿨습니다. 시물 결정론 자체는 기본 스위트의 상시 게이트로 남아 있습니다

(`tests/determinismGate.test.ts` 가 같은 시드를 두 번 돌려 상태 해시가 원소 단위로 같은지 검사합니다).

2. AI 디렉팅 파이프라인 — 사람은 무엇을 했는가

2-1. 지시문 계층 — AI가 매번 백지에서 시작한다는 전제

AI 세션은 이전 세션을 기억하지 못합니다. 그래서 사람의 판단을 문서로 굳혀 두고, 새 세션이 시작될 때 자동으로 주입되게 하는 계층을 만들었습니다.

계층	위치	담는 것
전역 규칙 13조	<code>~/.claude/CLAUDE.md</code>	프로젝트를 가리지 않는 작업 규약. 전면 한글, 병렬 세션은 워크트리로 격리, main 직접 push 금지, 계정 종속 MCP는 프로젝트 스코프로 격리 등
프로젝트 규약 5절	리포 <code>CLAUDE.md</code>	이 리포에서만 참인 검증 규약과 함정
도메인 원장	리포 <code>CONTEXT.md</code> (99KB)	용어와 도메인 모델의 단일 정보
결정 기록 55건	<code>docs/adr/</code>	되돌리기 어려운 결정과 그 이유
레인 문서 161건	<code>.omc/</code>	계획 66 · 조사 52 · 인수인계 43

전역 규칙 13조는 전부 사고를 겪고 추가한 것입니다. 예를 들어 7조(병렬 세션은 반드시 git worktree로 격리)는, 두 세션이 같은 작업 디렉터리를 만져 한쪽이 다른 쪽의 브랜치를 이름까지 바꿔 머지해 버린 사고 직후에 생겼습니다. 규칙만으로는 안 지켜져서 나중에 감지 혹까지 붙였습니다.

2-2. 설계 확정 — 딥 인터뷰와 ADR

개발 첫날 AI와 딥 인터뷰를 12~14라운드 돌려 게임 설계 문서(GDD)를 확정했습니다. AI가 모호한 지점을 소크라테스식으로 추궁하면 사람이 결정하고, 그 결과가 `docs/GDD.md` 로 남습니다. 이 방식은 이후에도 큰 기능마다 반복했고, 산출물이 `.omc/specs/` 에 15건 쌓여 있습니다.

되돌리기 어려운 결정은 ADR 55건으로 고정했습니다. 여기에는 폐기와 개정 관계까지 적습니다.

`docs/adr/AGENTS.md` 가 색인 역할을 해서, 어떤 ADR이 어떤 ADR을 개정했는지 한눈에 보입니다. AI가 폐기된 결정을 근거로 코드를 쓰는 것을 막기 위한 장치입니다.

2-3. 실행 — 워크트리로 격리한 병렬 레인

작업은 git worktree로 격리한 병렬 레인에서 진행했습니다. 여러 AI 세션이 동시에 서로 다른 기능을 각자 브랜치와 PR로 밀고 나갑니다. 스킬 210종을 배선할 때는 7개 레인이 동시에 돌았고, UI를 화면 단위로 개편할 때도 화면마다 레인을 갈랐습니다.

병렬에는 병렬의 대가가 있었습니다. 7개 레인이 전부 같은 파일(world.ts)의 단일 수렴점을 건드리는 구조라, 공통 기반을 선행 커밋으로 빼지 않으면 병렬의 이득이 머지 비용으로 전부 사라진다는 것을 조사 단계에서 발견했습니다. 그래서 기반을 먼저 한 번에 깔고 레인을 갈랐습니다.

2-4. 역할 분리 — 작성자는 자기 결과를 승인할 수 없다

한 레인 안에서도 역할을 나눈 서브에이전트를 씁니다. 오케스트레이션 계층으로 쓴 oh-my-claudecode가 19종의 에이전트 타입을 제공하고, 그중 이 프로젝트에서 상시로 쓴 것은 다음입니다.

에이전트	맡긴 일
explore	코드베이스 조사. 넓게 훑고 결론만 가져옵니다
planner · architect	계획 수립과 아키텍처 판단
executor	실제 구현. 무거운 건은 Opus로 올려 씁니다
critic	계획과 결과의 다각도 비평
code-reviewer · security-reviewer	품질과 취약점 검토
verifier	완료 주장에 증거가 붙었는지 확인
debugger · tracer	근본 원인 추적

핵심은 작성자가 자기 결과를 스스로 승인하지 못하게 한 것입니다. 별도 컨텍스트의 비평자가 결함을 적발하고, 비평이 반복해서 잡아낸 결함 패턴은 문서로 쌓아 다음 레인의 지시문에 되먹였습니다.

이 구조에서 실제로 사고도 겪었습니다. 팀 컨텍스트에서 읽기 전용 리뷰어를 부르면 리뷰어가 결과를 최종 텍스트로만 반환하고 보고 채널을 쓰지 않아, 리드 입장에서는 무응답으로 보이지만 리뷰는 이미 끝나 있는 상황이 생깁니다. 이때 실제 결함(off-by-one)을 담은 지적이 유실된 채로 머지된 적이 있어서, 이후에는 리뷰어를 팀 밖에서 띄우고 무응답이면 기록을 직접 확인하도록 규칙을 고쳤습니다.

2-5. 작업 칩 — 세션을 넘기는 단위

한 세션에서 발견했지만 지금 고치면 범위가 커지는 일은, 그 자리에서 고치지 않고 독립 작업으로 떼어 냅니다. 떼어 낼 때는 이 대화를 모르는 사람이 읽어도 착수할 수 있도록 파일 경로와 맥락을 전부 담습니다. 26일 동안 이렇게 떼어 낸 작업이 136건입니다. 각 칩은 새 워크트리에서 새 세션으로 시작해 PR까지 갑니다.

2-6. 실패를 지식으로 굳히는 2층 구조

같은 실수를 두 번 하지 않게 하는 것이 이 프로젝트에서 가장 큰 효율이었습니다. 실패를 겪을 때마다 원인과 재발 방지책을 문서로 굳히고, 범위에 따라 두 층에 나눠 놓았습니다.

프로젝트 한정 지식 (리포 .omc/skills/, 6건)

문서	굳힌 함정
<code>pixi-canvas-texture-expertise</code>	Pixi v8에서 캔버스를 텍스처로 만들 때 전용 소스 타입을 안 쓰면 GPU 텍스처가 경고 없이 빈 채로 남는다
<code>probability-gate-test-vacuity-expertise</code>	확률 게이트를 도입하면 기존 고정 픽스처 테스트가 조용히 공허해진다
<code>relative-security-bound-expertise</code>	상대 기준 방어는 분모만 내려도 깨진다. 밸런스 변경이 코드 없이 보안 기준을 무너뜨리는 경로
<code>sql-redefinition-observability-expertise</code>	<code>create or replace function</code> 이 뒤따르는 권한 설정을 조용히 지운다. 증상이 0인 보안 회귀
<code>render-screen-verification-expertise</code>	렌더 결함은 합성된 화면에서만 측정된다
<code>planet-blitz-supabase-deploy-workflow</code>	마이그레이션-Edge Function 배포 절차와 함정 4종

프로젝트 횡단 지식 (`~/ .claude/skills/omc-learned/`, 9건) — 다른 프로젝트에서도 밋을 함정을 여기 둡니다. PowerShell 혹은 UTF-8 BOM 문제, 파생값 원장 보정, Supabase Edge Function 번들 크기 문제 같은 것들입니다.

여기에 더해, 세션이 끝날 때마다 무엇이 틀렸고 왜 틀렸는지를 메모리 문서로 남겨 다음 세션에 주입했습니다. "파이프가 exit code를 덮어 거짓 초록을 만든다" 같은 기록인데, 이 되먹임이 26일 완주의 실질적인 동력이었습니다.

2-7. 세션 경계를 지키는 혹 5종

AI 세션이 시작될 때 자동으로 도는 PowerShell 혹을 다섯 개 걸어 두었습니다. 사람이 매번 확인하지 않아도 되게 하려는 것입니다.

혹	하는 일
<code>concurrent-session-warn</code>	같은 워크트리를 여러 세션이 동시에 만지는지 감지해 경고합니다
<code>session-auto-ffpull</code>	리모트가 앞서 있으면 fast-forward만 당깁니다. 미커밋 변경이 있거나 로컬이 앞서 있으면 당기지 않고 알리기만 합니다
<code>link-shared-env</code>	새 워크트리에 공유 설정을 연결합니다. 없으면 게임이 통째로 오프라인으로 떨어집니다
<code>reap-orphan-omc-runners</code>	비정상 종료로 남은 고아 프로세스를 회수합니다
<code>sync-quiz-mirror</code>	다른 프로젝트의 스킬 목록이 섞여 들어오는 것을 막습니다

git 쪽에도 `post-checkout` 혹을 걸어, 새 워크트리를 만들면 의존성 설치와 환경 파일 복사가 자동으로 되게 했습니다.

2-8. 사례 — 산출물 대신 채점 기준을 고쳤다

디렉팅이 실제로 어떤 일이었는지 한 건으로 보이겠습니다.

전체 테스트 스위트에서 12건이 상시 빨간 채로 방치돼 있었습니다. 원인을 파 보니 단언이 봇이 런을 완주하는지에 걸려 있어서, 밸런스 패치로 피격 피해 배수가 바뀐 순간 드랍과 경험치 배선을 재려던 테스트들이 썰 것을 못 재고 있었습니다. 개별 테스트를 손보는 대신 원칙을 세웠습니다. 단언이 "봇이 이길 수 있는가"에 의존하면 게이트에서 내린다는 것입니다(ADR-0051). 그 축을 내리되 배선 증명은 내구 파일럿과 최소 톱으로 다시 써서 살렸습니다. 지우고 도망친 것은 아닙니다. 그 결과 스위트가 241초에서 55초로 줄고 main이 전량 초록이 됐습니다.

ADR도 채점 대상이었습니다. ADR-0050은 구현에 착수하기 직전 전수조사에서 자신의 전제 셋이 코드와 어긋난다는 사실이 드러나, 정정 절을 달고 집행됐습니다. 그대로 밀어붙였다면 아무것도 막지 못하는 문을 잠글 뻔했고, 결정은 유지하되 표적만 다시 잡았습니다. 이런 자기 정정 기록을 지우지 않고 남겨 두는 것이 AI 세션들의 같은 실수를 막아 주었습니다.

2-9. 검증 게이트 — 통과해야 다음으로 간다

- **검증 규약을 지시문으로 못박았습니다.** 편집 중에는 짝 테스트, 커밋 전에는 변경분 테스트, PR 전에는 전체 스위트와 타입 검사와 린트를 돌립니다. 리포 `CLAUDE.md` 에 적어 두어 모든 AI 세션에 자동으로 주입됩니다.
- **결정론 게이트**는 대표 행성과 빌드 조합을 같은 시드로 두 번 돌려 해시가 맞는지 상시 검사합니다. 골든 상수와 대조하는 대신 자기 자신과 대조하므로 유지비 없이 재현성을 지킵니다.
- **CI**는 GitHub Actions에서 타입 검사를 통과해야만 Pages 배포가 나가도록 묶었습니다. 타입 오류가 있으면 배포 자체가 차단됩니다.

3. 26일의 기록 — 게임이 어떤 지시로 완성돼 왔는가

아래는 제가 직접 입력한 프롬프트 원문을 시간순으로 배열한 것입니다. 도구나 환경에 관한 것은 빼고 게임 자체의 방향을 정한 것만 골랐습니다. 이 순서가 곧 이 게임이 만들어진 순서입니다.

1단계 (7/15~7/18) — 골격을 세우고, 내가 직접 만질 손잡이를 요구하다

첫 주에는 마일스톤 계획을 세우고 시뮬레이션 코어와 기본 루프를 올렸습니다. 이 시기에 제가 건 요구 중 나머지 26일을 좌우한 것이 하나 있습니다. AI가 만든 것을 **제가 직접 플레이해서 판단할** 수단을 달라는 것이었습니다.

내가 의도한 하네스는 웹브라우저에 게임 화면을 띄워주면 여러 톨로 내가 게임을 직접 돌려보면서 여러가지 테스트를 할 수 있는 톨이었어.

이 요구로 개발용 하네스가 생겼고, 이후 모든 밸런스 판정과 시각 판정이 이 위에서 이뤄졌습니다. 같은 시기에 작업 순서도 다시 잡았습니다. 코드 작업을 먼저 하고, 그다음이 제가 직접 하는 플레이테스트이고, 외부 SDK 연동은 미정으로 미뤘습니다. 만든 뒤에 반드시 사람이 만져 본다는 순서를 이때 고정한 셈입니다.

2단계 (7/19~7/20) — 첫 UI 패스, 그리고 지적을 지식으로 바꾸다

기지와 격납고를 비롯한 화면에 처음으로 통일된 UI를 입혔습니다. 이 단계는 제가 화면을 보고 지적하면 AI가 고치는 반복이었습니다.

- 주무기 칸에 있는 콘텐츠도 밖으로 빠져나가고 있어.
- 기체 스탯 안에 있는 글자들이 너무 뽁뽁하고 살짝 깨지는 감이 있어.
- 좌하에 hp바와 lv 바가 겹쳐.
- 전체적으로 원래 목업의 느낌과 조금씩 다른거 같아. 목업에서 확인했던게 다 엉망이 된거 같아. 다시 확인해봐.

여기서 방식을 바꾼 계기가 나왔습니다. 같은 종류의 지적을 두 번 하게 되자, 고치는 것으로 끝내지 말고 재발을 막으라고 요구했습니다.

밑에 4개의 각 칸들의 제목이 테두리와 너무 붙어 있어. 이건 전에도 지적한거 같은데 같은 실수 반복하지 않게 해줘.

ok. 딱 좋아. 이번에 UI 작업한 UI셋을 한번 적용할 수 있는 스킬과 UI에 적용하면서 발생한 문제점들을 교훈 삼아서 UI에 적용을 잘할 수 있게 하는 스킬을 만들어서 (...) 잘 만들어줘.

지적을 문서로 굳혀 다음 세션이 자동으로 읽게 만드는 §2-6의 학습 구조가 여기서 시작됐습니다.

3단계 (7/21) — 만들기 전에 운영 가능한지 먼저 물었다

콘텐츠를 더 붙이기 전에 서비스가 성립하는지부터 확인했습니다.

이렇게 최적화를 한다는 가정아래 하루에 만명정도의 유저가 한시간 정도 플레이를 한다고 하면 supabase를 무료로 유지 가능할까?

이 질문의 답이 이후 서버 설계를 규정했습니다. 런 중 통신을 하지 않고 정산 시점에만 서버를 때리는 구조, 리플레이를 서버에 상시 보관하지 않는 정책이 전부 여기서 나왔습니다.

4단계 (7/29~7/30) — 2D 게임을 시각적으로 끌어올리다

기본 루프가 돌아가기 시작하자 시각 품질로 넘어갔습니다. 이때 품질 기준을 말로 정의하는 대신 **판정 방법으로** 정의했습니다.

카르곤 행성의 배경을 AAA 게임 수준으로 만들어라. (...) 하위 에이전트들을 여러 명 배치하여 각 에이전트가 개별적으로 게임을 완벽하게 만들도록 하세요. (...) 이 에이전트는 매우 엄격한 비평가 역할을 해야 하며, AAA급 퀄리티에 만족한 후 다음 항목으로 넘어가야 합니다. 각 하위 에이전트가 실제 AAA 게임과 비교했을 때 그래픽 품질에 완전히 만족할 때까지 작업을 계속해야 합니다. 심지어 눈을 감고 두 게임을 나란히 비교하여 어느 쪽이 더 나은지 판단할 수 있어야 합니다.

같은 형식을 플레이어 기체와 적, 해저드에도 적용했습니다. 이어서 3D를 도입했는데, 전면 도입이 아니라 검증 순서를 지정했습니다.

3d 모델을 통해서 보스가 다양한 연출을 통해 다양한 애니메이션을 보여줬으면 해. 그리고 이게 잘되는지 확인되면 플레이어 기체에도 똑같이 적용하고 싶어. 이럴때는 어떤 방식이 가장 효과적일까? 보스는 페이즈마다 다른 애니메이션을 보여야 한다. 이를 충족하면서 2단계까지 진행하고 결과를 나에게 보여줘

보스 6종으로 먼저 확인한 뒤 플레이어 기체로 넘어갔습니다. 그 사이에 결과가 마음에 안 들면 되돌리는 지 시도 있었습니다.

이전에 플레이어 기체의 그래픽을 업그레이드 하는 작업을 했었는데 결과가 마음에 안들어. 기체 뒷부분에 불꽃 부분만 남기고 다른건 원래대로 돌려주고 기체 주위에 둥그렇게 파랗게 효과가 들어가는데 이걸 뭐야?

5단계 (8/1~8/2) — 타이틀 연출과 화면 전면 개편, 그리고 판정 권한을 되찾다

타이틀과 인트로를 먼저 손봤습니다. 연출 지시는 결과물이 아니라 **느낌**으로 줬습니다.

함선이 지금 플레이어가 있는 곳에서 가운데 있는 행성으로 날아간다는 느낌이 들어야 해. 그래서 함성 뒷면이 보이는 느낌으로 지금 보다 좀 더 작아져도 돼.

그리고 나니 나머지 화면이 초라해 보였고, 여기서 게임 전체의 시각 기준선이 정해졌습니다.

타이틀과 인트로를 멋있게 작업했더니 상대적으로 실제 게임 화면이 너무 별로다. 특히 기지 화면은 너무 아마추어 느낌이 난다. 일단 기지 화면을 아래 내용에 맞추어서 작업을 하고 결과를 본후 다른 화면에도 적용한다.

AAA급의 게임 품질로 디자인을 해주고 메인 화면과 톤앤매너를 맞춰줘. 서브에이전트들을 분산 배치하고 (...) 별도의 서브에이전트가 시각적으로 검토하여 AAA급 품질인지 확인해야 합니다. (...) 말 그대로 눈을 감고 두 화면을 나란히 비교하여 어느 쪽이 더 나은지 판단할 수 없는 수준까지 나와야 합니다.

기지 화면을 먼저 개편하고 확인한 뒤, 격납고·연구소·정제소·방어 사령부·관제탑·기록 보관소·지시 수신소·성계 지도까지 같은 계약을 이어 붙였습니다. 화면마다 레인 계약 문서와 레이아웃 불변식 테스트(화면당 30건 안팎)가 함께 나왔습니다.

이 단계에서 방식을 한 번 더 바꿨습니다. 비평 에이전트가 세운 시각 지표를 제가 화면을 보고 통째로 뒤집는 일이 반복되자, 판정 권한을 회수했습니다.

앞으로 비평은 니가 하지 말고 내가 직접 보고 수정 사항을 알려줄게.

이후 작업 지시문에는 "비평 서브에이전트를 자동으로 돌리지 마라. 판정 기준은 스크린샷이고 사용자가 최종 판정자다"라는 문구가 고정으로 들어갔습니다. 프롬프트 한 줄이 워크플로를 바꾼 자리입니다.

6단계 (8/4~8/5) — 지형과 장기 곡선

전장에 구조물을 넣으면서, 지형을 난이도 손잡이로 쓰지 말라고 못박았습니다.

각 행성별로 벽은 한가지씩만 사용하고 겹치지 않게 해주고 난이도와 상관없이 좀 길게 해줘. 모양 상관없이 프리팹당 최소 7개 이상은 이어져있게 해줘. 난이도 신경쓰지마.

"난이도 신경쓰지마"가 중요한 지시였습니다. 벽이 구조물이 되자 붓의 사선이 자주 막혀 경제 지표 두 개가 밴드를 벗어났는데, 이때 벽을 되돌리는 대신 적 축과 보상 축에서 복구하도록 방향을 잡을 수 있었습니다. 지형은 제가 고른 인상이고 경제는 별도 축이라는 구분입니다.

장기 성장 곡선은 설계 문서로는 안 보여서 직접 굴러 확인했습니다.

질문이 있어. 게임에 처음 들어온 1일차 플레이어일 경우와 방금 비행체 1기를 퇴역 시키고 레벨1 비행체를 새로 받은 플레이어가 30일 연속으로 들어왔을 경우 30일동안 받는 보상을 시뮬레이션해서 보여줘.

7단계 (8/8) — 출시 전 밸런스, 직접 플레이하고 지적하는 연쇄

마지막 주는 제가 직접 플레이하고 그 자리에서 지시를 넘기는 반복이었습니다. 하루 안에 일어난 연쇄를 순서대로 옮깁니다.

먼저 한 판의 템포를 정했습니다.

행성별 런타임을 보스전까지 1분 30초로 만들고 파워업 3개 중에 하나고르는거 수치들이 너무 높아. 특히 공격력 올리는 부분은 아무리 높아도 10% 이하로 해줘.

여기서 나온 90초가 이후 모든 밸런스 계측의 기준선(`RUN_SECONDS_PAR`)이 됐습니다. 클리어초 게이트의 밴드가 이 값의 0.63~2.0배로 잡혀 있습니다.

플레이해 보니 보스가 너무 빨리 죽었습니다.

하나만 더 고치자. 보스의 HP가 낮아서 10단계와 20단계에서 테스트 했을 때 너무 빨리 죽었어. 거의 3초안에 죽은거 같아. 30초 정도는 버틸 수 있게 수정해줘.

이 한 건 때문에 보스 처치 시간을 직접 재는 계측 CLI가 새로 생겼습니다. 체감 신고가 계측 도구를 하나 만들어 낸 사례입니다.

다음은 화면 밀도였습니다. 숫자를 지정하되 난이도는 고정하는 형태로 요구했습니다.

적들이 지금보다 30% 정도 더 많이 나왔으면 좋겠고 적들의 탄환이 지금보다 30% 정도 더 많았으면 좋겠어. 그런 상태로 난이도가 비슷하게 유지 됐으면 좋겠어. 이렇게 만들고 다시 한번 플레이 해볼게

"둘을 올리되 셋째는 고정"이라 AI가 임의로 균형을 잡을 수 없고 반드시 계측으로 확인해야 합니다. 실제로 이 요구 때문에 탄이 46% 늘어도 무입력 붓의 피격량은 거의 안 움직인다는 사실이 드러났고, 붓은 카이팅을 해서 원래 잘 안 맞기 때문이었습니다. §4-4에서 말하는 "붓이 못 재는 축"의 대표 사례가 여기서 나왔습니다. 다시 플레이한 뒤에는 이렇게 이어졌습니다.

화면에 나오는 적과 탄수는 좋아. 대신 좀 어려우니까 내 기체의 공격력을 15% 방어력을 20% 올려줘. 그리고 나서 다시 해보자

플레이하다 장비 설계의 결함도 드러났습니다.

그럼 장비 설계가 잘못됐어. 어픽스가 하나도 없어서 입으나 마나인건 말이 안돼. 노말은 어픽스 1개. 매직은 2~3 레어는 3~6으로 하자

이 지시가 그대로 현재 구현입니다. 노말 1개, 매직 2~3개, 레어와 유니크 3~6개이고, 코드 주석에 "노말이 입으나 마나인 등급이라 스타터킷이 매직을 쓸 수밖에 없었다"는 경위가 남아 있습니다.

초반 진입 난이도는 두 번에 걸쳐 조정했습니다. 여기서 중요한 것은 **측정 기준과 제품 기본값을 분리**하고 요구한 점입니다.

현재 초반 시작시 시작 장비를 제공하고 있어. 확인해봐. 물론 그로 인해 밸런스 체크가 흔들리면 안되니까 밸런스 체크는 일단 장비가 없는 걸로 하고 초반에 사용자가 쉽게 시작할 수 있을 정도의 장비만 제공하는 걸로 다시 생각해보자

근데 초반 장비를 레어로 지급하는건 좀 너무 많이 주는거 같아. 아주 기본적인 가장 저급의 장비만 지급하는 걸로 바꾸자.

마지막으로, 이 반복 자체를 절차로 만들었습니다.

이렇게 하자. 니가 할 수 있는건 다 하고 나서 마지막에 내가 3번의 플레이를 하고 각 플레이를 할 때마다 의견을 주고 또한 니가 결과 내용을 본다. 그러고 나서 그 결과를 바탕으로 다시 조정할 내용을 니가 반영한다.

AI가 봇으로 켈 수 있는 데까지 재고, 그다음 사람이 세 판을 직접 하고, 그 체감과 봇의 표를 함께 놓고 다시 조정한다는 분업입니다. 이것이 26일 동안 도달한 최종 형태이고, 다음 절에서 그 기계 쪽 절반을 설명합니다.

부록 — 위 지시들의 원문 인용에 대해

인용문은 세션 기록에서 그대로 옮겼습니다. 오타와 구어체를 포함해 손대지 않았고, 길이가 긴 것은 (...)로 줄였습니다. 제가 작성해 다른 세션에 넘긴 작업 지시서 136건은 사람이 타이핑한 프롬프트가 아니므로 이 절에서 제외했습니다.

4. 밸런스 검증 — 봇과 사람의 분업

밸런스는 이 프로젝트에서 AI에게 가장 위임하기 어려운 축이었습니다. 최종적으로 도달한 형태는 "기계가 재고, 사람이 판정한다"입니다.

4-1. 밸런스 테스트를 게이트에서 내렸다

원래는 밸런스 계측이 자동 테스트 스위트 안에 있었습니다. 그런데 그 테스트들이 실제로 재는 것은 게임의 밸런스가 아니라 **봇의 실력**이었습니다. 봇 클리어율 61.4%가 사람에게 무엇을 뜻하는지 아무도 모르는데, 그 숫자가 빨개졌다고 빌드를 막고 있었습니다. 결국 12건이 상시 빨간 채로 방치됐고, 깨진 채 방치된 게이트는 게이트가 아니었습니다.

ADR-0051에서 판정 규칙을 세웠습니다. 단언이 "봇이 이길 수 있는가"에 의존하면 게이트에서 내리고, "값이 흘러가는가"에만 의존하면 남기되 완주가 아니라 최소 틱으로 증명한다는 것입니다. 그리고 계측 코드를

아예 tests/ 그룹 밖(bench/)으로 옮겨, 테스트 러너가 구조적으로 수집할 수 없게 만들었습니다. 규율을 사람의 의지가 아니라 디렉터리 구조로 강제한 것입니다.

이 판단으로 스위트가 241초에서 55초로 줄었고, 밸런스는 손으로 돌리는 CLI의 몫이 되었습니다.

4-2. 10분 안에 끝나는 밸런스 하네스

밸런스를 손으로 돌리게 됐으니, 한 번 돌리는 데 시간이 오래 걸리면 아무도 안 돌리게 됩니다. 그래서 요구 사항을 두 개 걸었습니다. **10분 안에 끝날 것**, 그리고 **새 행성·새 기체·새 장비·새 기능이 들어와도 하네스를 거의 안 고쳐도 될 것**입니다.

`pnpm balance` 한 번이면 난이도 곡선, 기체 간 편차, 행성 간 편차, 경제 지표를 동시에 재고 게이트 판정까지 냅니다. 10분 안에 끝나는 이유는 장치 세 개의 공급입니다.

1. **병렬 실행** — 시물을 단일 번들로 묶고 워커 스레드로 (코어 수 - 2)개를 돌립니다. 16코어 머신에서 14 배입니다.
2. **라운드 단위 시드 확장과 예산 감시** — 한 라운드는 모든 셀에 시드를 하나씩 굴립니다. 다음 라운드를 돌릴 시간이 있는지 매번 판단하므로 예산 초과가 구조적으로 불가능하고, 어디서 멈춰도 표본이 모든 셀에 고르게 깔립니다.
3. **머신 실측 캘리브레이션** — 시작 직후 작은 격자로 이 머신의 틱당 비용을 재서 감당 가능한 셀 수를 계산합니다. 격자가 크면 축을 줄이되 양 끝은 보존합니다. 하드웨어 상수를 리포에 박아 두지 않기 위한 설계입니다.

새 콘텐츠가 들어올 때 하네스 수정량은 0입니다. 행성·기체·장비·스킬·만렙은 전부 카탈로그에서 파생되기 때문입니다. 새 지표를 추가할 때만 정의 한 줄을 넣으면 되고, 목표 밴드를 함께 적으면 게이트가 자동으로 생깁니다. 이 계약은 테스트가 지키고 있습니다.

4-3. 무엇을 재는가 — 격자와 지표

측정 단위는 격자 한 칸이고, 한 칸은 (행성, 기체, 표준 레벨)입니다. 행성 6 × 기체 7 × 레벨 10 = 420셀이 기본 격자입니다. 각 셀을 표준 장비와 표준 스킬 투자로 고정한 뒤 결정론 봇이 시드를 여러 개 굴립니다. 난이도 곡선, 기체 편차, 행성 편차는 전부 같은 격자를 다르게 접은 결과입니다.

게이트가 걸린 지표는 다섯 개입니다.

지표	목표 밴드	판정 범위	근거
클리어율	60~80%	레벨별	ADR-0037 표준 빌드 합격선
클리어초	60~190초	전체	런 par 95초의 0.63~2.0배
타임아웃률	0~2%	레벨별	구조 건전성(18,000틱 상한)
런 내 레벨업	5~8회	전체	ADR-0036
완주 런당 장비 유입	2~3개	전체	보스 1 + 엘리트 1.5

게이트가 없는 진단 지표도 열 개 넘게 같이 냅니다. 보스 도달 시간, 미수거 전리품, 후방 압박, 맵 상실, 정 화율, 지형·접촉 피해 비중 같은 것들입니다. 이들은 판정을 하지 않고 처방을 가릅니다. 클리어율이 낮다는 사실만으로는 무엇을 고칠지 모르지만, 후방 압박이 런 시간의 42%라면 원인이 다른 곳에 있다는 것을 알 수 있습니다.

밴드 운용에서 배운 것 두 가지를 문서에 못박아 두었습니다.

- **모집단을 잘못 잡으면 지표가 거짓말을 합니다.** 장비 유입을 전체 런 평균으로 재던 때에는, 승률이 낮아 질수록 장비 유입 게이트가 자동으로 빨개졌습니다. 완주 런 평균으로 고쳤습니다.
- **전체 평균이 국소 이상을 가립니다.** 타임아웃률이 전체 0.8%로 초록인 동안 Lv100만 10.6%, 그중 니플헤임은 44.8%였습니다. 그래서 지표마다 판정 범위(전체냐 레벨별이냐)를 따로 지정합니다.

4-4. 봇으로 재고, 봇의 한계를 문서에 적는다

측정에 쓰는 오토파일럿은 순수 결정론 입력 봇입니다. 월드 상태만 읽어 한 틱의 입력을 만들고, 난수도 시계도 쓰지 않습니다. 조작 결정은 우선순위 분기로 되어 있습니다. 강제 스크롤 무대에서는 화면 밖으로 밀리지 않는 것이 최우선이고, 그다음이 적탄 회피, 진행에 걸린 목표물 파괴, 전리품 수거, 사선 확보, 카이팅 순입니다.

중요한 것은 이 봇이 **최적 플레이를 하지 않는다**는 점이고, 그 사실을 문서에 자인으로 적어 두었습니다. 봇이 못 재는 축은 이렇습니다.

- 액티브 스킬을 쓰지 않습니다(측정 전용 파일럿만 씁니다)
- 전리품 수거 반경이 제한돼 있습니다
- 파워업 선택을 최적화하지 않습니다
- 침공(PvP)과 의뢰는 이 격자에 아예 없습니다

그래서 판정 대상은 절대값이 아니라 **축 간 상대 난이도와 밴드 이탈**입니다. 밸런스 문서에 "봇 밴드를 절대 기준으로 쓰지 않는다"를 선언으로 박아 두었고, 다음 레인이 이걸 모르고 밴드 위반이라고 부르면 그것은 오독이라고 적었습니다.

4-5. 계측기가 먼저 고장난다

이 프로젝트에서 반복해서 겪은 것은, 밸런스가 이상해 보일 때 원인이 게임이 아니라 계측기에 있는 경우가 잦다는 사실입니다.

- 아르케(강제 스크롤 레이싱) 클리어율이 1.7%로 나왔습니다. 게임이 어려운 것이 아니라 봇이 화면 창의 존재를 몰라 뒤로 후퇴하고 있었습니다. 진단 지표 하나가 "후방 압박이 런 시간의 42.8%"라고 알려 주어서 잡았습니다. 처방은 밸런스 조정이 아니라 계측기 수리였습니다.
- 니플헤임(추격)에서 승리한 런 422건 전부가 "보스 교전 0회"로 집계됐습니다. 이 모드는 표적이 런 시작부터 존재하고 한동안 무적이라, 보스 도달 판정의 정의가 이 모드에만 안 맞았던 것입니다.
- 레버를 4배로 올렸는데 결과가 10%만 움직인 일이 두 번 있었고, 둘 다 원인이 같았습니다. 그 대상이 애초에 자동 조준 대상이 아니었습니다. 그래서 "레버가 연결돼 있는지 먼저 확인하라, 입력 대비 출력 크기부터 보라"를 절차에 넣었습니다.

4-6. 사람이 판정하는 자리

밸런스 하네스는 판정을 내지만 처방을 내지 않습니다. 처방이 밸런스 변경이면 사람이 정합니다. 그 판정은 대화에 흘러보내지 않고 표로 리포에 남깁니다. 실제 기록의 한 대목입니다.

축	제시된 선택지	판정
단계 16~19 밴드 아래	유지 / 적 피해 완화 / 공비 조정 / 재플레이 후 결정	유지. 단계 20 = 65초를 지킨다
단계 20의 계측기 불연속	눈금 수정 / 주석만 / 전용 계측 신설 / 안 고친다	안 고친다
침공 상한 초과	원인부터 가르다 / 난이도를 올린다 / 그대로 둔다	그대로 둔다
단계 1~11 적 피해 배수	재플레이 후 결정 / 유지 / 완화 / 원복	유지

그리고 판정 옆에 "왜 이 판정이 위험한가"를 같이 적었습니다. 예를 들어 마지막 항목은 봇이 가장 못 재는 축을 봇으로 정한 값이므로, 출시 후 초반 이탈률이 튀면 가장 먼저 의심할 값이라고 적어 두었습니다.

사람이 직접 플레이해서 판정하는 절차도 워크플로에 넣었습니다. 게임 안 개발 하네스가 현재 런 상태를 JSON 스냅샷으로 뱉는데, 제가 직접 플레이한 뒤 그 스냅샷을 그대로 붙여 넣고 체감을 덧붙이는 방식입니다. "50레벨은 좀 쉬웠다. 근데 촉매 쓸 거 생각하면 괜찮은 거 같다" 같은 문장과 해시·시드가 함께 남으므로, 판단의 근거가 되는 런을 나중에 그대로 재현할 수 있습니다. 결론은 "지금보다 공격력 10% 올리고 확정하자" 같은 형태로 냅니다.

4-7. 개별 계측 CLI 11종

전체 하네스와 별개로, 단일 축을 좁게 재는 CLI를 필요할 때마다 만들었습니다.

CLI	재는 것
<code>runCurve</code>	표준 빌드의 레벨별 곡선. 촉매를 넣은 런과 안 넣은 런을 같은 시드로 돌려 배수를 내는 모드도 있습니다
<code>invasionBands</code>	침공 난이도 밴드(기본 24시드)
<code>commissionBands</code>	의뢰 다구간 난이도(기본 96시드, 9밴드)
<code>nominalPower</code>	기체 7종의 명목 파워. 시물을 한 틱도 안 돌리는 닫힌 식이라 수 밀리초
<code>gearLevelCurve</code>	레벨 성장 곡선. 합격선이 "Lv100이 Lv50보다 약할 확률"입니다
<code>gearDps</code>	불사 더미를 세워 놓고 재는 실패 화력
<code>bossTtk</code>	보스 처치 시간. 위 ④번 지시로 생겼습니다
<code>incomingDps</code>	초당 피격량
<code>contactPilot</code>	몸으로 부딪혀야 얻는 전리품을 재기 위한 계측기
<code>phantomCycleProbe</code>	팬텀 은신 사이클 실측
<code>dumpSkills</code>	스킬 210종 매니페스트 덤프

5. 사용한 도구와 플러그인, 그리고 사용 방식

5-1. Claude Code (Anthropic) — 본체

게임 코드·테스트·문서 전량을 작성했습니다. 이 프로젝트에서 쓴 주요 설정입니다.

- 모델은 Opus를 긴 컨텍스트 모드로 고정하고, 추론 강도를 높게 두었습니다.
- 에이전트 팀 기능을 켜서 한 세션이 여러 워커를 이름으로 관리할 수 있게 했습니다.
- 안 쓰는 내장 스킬 14종은 명시적으로 꺾습니다. 스킬 목록이 컨텍스트를 상시로 잡아먹기 때문에, 이 프로젝트에 안 쓰는 것을 꺼서 세션당 고정 지출을 줄였습니다.
- 상태 표시줄에 커스텀 HUD를 붙여 현재 모드와 진행을 항상 보이게 했습니다.

5-2. oh-my-claudecode — 멀티에이전트 오케스트레이션 계층

에이전트 19종과 스킬 41종을 제공하는 플러그인입니다(4.15.7 사용). 이 프로젝트에서 실제로 돌린 워크플로는 다음입니다.

스킬	사용 방식
<code>/deep-interview</code>	모호한 요구를 소크라테스식 문답으로 좁힙니다. 모호도가 임계 아래로 떨어져야 스펙이 나옵니다. 산출물 15건
<code>/ralplan</code> · <code>/plan</code>	계획을 세우고 Planner/Architect/Critic이 합의할 때까지 돌립니다
<code>/autopilot</code>	요구를 스펙으로 분해하고 계획·구현·QA·검증을 자동으로 태웁니다
<code>/ultrawork</code> · <code>/ralph</code>	완료 조건을 만족할 때까지 멈추지 않는 실행 모드
<code>/team</code>	여러 워커를 동시에 띄워 공유 작업 목록을 처리합니다
<code>/goal</code>	조건이 성립할 때까지 세션이 멈추지 못하게 겁니다. 밸런스 반복 수정처럼 "될 때까지" 돌려야 하는 일에 썼습니다
<code>/learner</code> · <code>/skillify</code>	이번 세션에서 배운 것을 재사용 가능한 스킬 문서로 굳힙니다. <code>.omc/skills/</code> 6건이 이 산출물입니다
<code>/verify</code> · <code>/visual-verdict</code>	완료 주장에 증거가 붙었는지, 화면이 실제로 그렇게 보이는지 확인합니다

5-3. PixelLab — 픽셀아트 생성 (자작 플러그인으로 감쌌)

기체·적·보스·아이콘·타일·UI까지 픽셀아트 573장을 생성했습니다. 다만 API를 그대로 쓰지 않고 **직접 플러그인을 하나 만들어**(`pixellab-forge`) 그 위에서 썼습니다. 생성물을 그냥 두면 나중에 같은 그림을 다시 못 만들고 재사용도 안 되기 때문입니다. 이 플러그인이 하는 일은 생성물마다 프롬프트·라이선스·해시를 인덱스에 기록하고, 캐시와 저장소를 세션 단위로 동기화하는 것입니다. 현재 인덱스에 974건이 쌓여 있습니다.

여기서도 함정을 하나 겪고 규칙으로 굳혔습니다. 캐시에만 넣고 저장소로 옮기지 않으면 플러그인을 재설치할 때 유실되는데, 실제로 32건이 미반영된 상태로 발견됐습니다. 그래서 세션 마감 체크리스트에 캐시와 저장소의 이미지 수를 대조하는 절차를 넣었습니다.

5-4. Meshy — 3D 모델 생성 (자작 플러그인으로 감쌘)

보스 3D 모델과 타이틀 연출용 모델을 생성했습니다. 마찬가지로 `meshy-forge` 라는 플러그인을 직접 만들어 썼는데, Meshy의 생성물 URL이 3일이면 만료되기 때문입니다. 만료 전에 내려받아 보관하고 검색할 수 있게 하는 것이 이 플러그인의 존재 이유입니다.

3D 도입은 검증 순서를 나눠서 진행했습니다(\$3 ⑧번 지시). 렌더 쪽에서는 Pixi v8에 3D 캔버스를 엮을 때 전용 소스 타입을 안 쓰면 텍스처가 경고 없이 빈 채로 남는 함정을 밟았고, 이 지식을 프로젝트 스킬로 굳혀 두었습니다.

5-5. Chrome DevTools MCP — 화면을 실제로 보고 판정

렌더 결함은 코드를 읽어서는 안 보이고 합성된 화면에서만 드러납니다. 그래서 실제 브라우저를 띄우고 스크린샷을 찍어 판정하는 경로를 따로 만들었습니다. 게임에는 개발용 하네스를 붙여 두었는데, 이건 원래 제가 이렇게 요구한 것입니다.

내가 의도한 하네스는 웹브라우저에 게임 화면을 띄워주면 여러 툴로 내가 게임을 직접 돌려보면서 여러가지 테스트를 할 수 있는 툴이었어.

하네스는 URL 파라미터 하나로 켜지고, 화면 이동·런 시작·빨리 감기·일시정지·프리셋 적용·상태 스냅샷 같은 조작을 노출합니다. 사람이 직접 플레이할 때도 쓰고, AI가 스크린샷 검증을 할 때도 씁니다. 결정론 시뮬이라 같은 시드로 같은 프레임을 다시 만들 수 있어서, 특정 장면을 재현해 찍는 것이 가능합니다.

병렬 워크트리마다 다른 포트를 쓰도록 개발 서버 구성을 네 개 등록해 두었습니다. 여러 세션이 동시에 하네스를 띄워도 포트가 충돌하지 않습니다.

5-6. Supabase MCP — 계정 종속 도구는 프로젝트 스코프로 격리

DB 스키마 조회와 마이그레이션 확인에 씁니다. 이 도구는 전역이 아니라 **프로젝트 스코프로**만 붙였습니다. 계정에 묶이는 도구를 전역에 두면 다른 프로젝트에서 엉뚱한 계정을 가리키게 되기 때문인데, 실제로 다른 프로젝트에서 그 사고를 겪고 나서 전역 규칙으로 만들었습니다.

서버 배포는 MCP에 의존하지 않고 PowerShell 스크립트로 절차를 고정했습니다. 마이그레이션 적용 스크립트 19종, 롤백 2종, 그리고 **서버 권위가 실제로 작동하는지 원격에서 증명하는 스크립트 5종**이 있습니다. 마지막 것이 중요한데, 클라이언트에서 조작을 시도했을 때 서버가 실제로 막는지를 적용 스크립트가 아니라 별도의 증명 스크립트로 확인합니다. 적용 스크립트는 자기가 만든 상태를 자기가 확인하는 향진이 되기 쉽습니다.

5-7. 비-AI 스택과 CI

TypeScript 5.7 · PixiJS v8 · three.js · Vite 6 · Vitest 2 · Supabase(Postgres + Edge Functions) · pnpm 11.

GitHub Actions 워크플로는 하나이고, main에 푸시되면 타입 검사를 포함한 빌드를 돌린 뒤 Pages로 배포합니다. 타입 오류가 있으면 배포가 차단됩니다.

6. 외부 에셋 · 오픈소스 출처

자산	출처 · 라이선스
BGM 6트랙	Juhani Junkala · congusbongus · Emma_MA (OpenGameArt) — 전부 CC0 1.0. 트랙별 출처 링크는 리포 <code>assets/audio/CREDITS.md</code>
효과음 9종	Kenney.nl 「Sci-Fi Sounds」·「Interface Sounds」 — CC0 1.0, 동일 문서에 명시
픽셀아트	Pixellab 생성물 — 상업 이용 가능 (약관: https://pixellab.ai/termsofservice)
3D 모델	Meshy 생성물 — 상업 이용 가능 (meshy.ai 이용약관)

음악만은 AI 생성물을 의도적으로 배제했습니다. 학습 데이터의 저작권이 불확실하다고 판단해서인데, 이 정책은 `assets/audio/CREDITS.md` 에 적어 두었고 전 트랙이 사람이 작곡한 CC0 음원입니다.

오픈소스 의존성은 전부 MIT입니다. `pixijs`, `three`, `@supabase/supabase-js`, `vite`, `vitest`, `eslint` 등이고 전체 목록은 `package.json` 에 있습니다. 게임 소스 자체는 독점 라이선스지만 리포가 공개돼 있어 열람은 자유롭고, 플레이는 배포 링크로 하시면 됩니다(「게임 소개 문서」 §2).

7. 다음 단계 — 이 방식이 아직 못 하는 것

26일 동안 확인한 것은 하나입니다. 기계가 판정할 수 있게 아키텍처를 설계하면, AI 산출량의 상한이 사람의 검토 속도에서 게이트의 처리량으로 옮겨 갑니다. 남은 병목은 셋인데, 셋 다 AI의 다음 단계가 풀어야 할 문제라고 봅니다.

- 재미는 아직 기계가 못 잹니다.** 결정론 봇으로 클리어율과 구간 틱은 재지만 재미있는지는 재지 못해서, 밸런스는 여전히 사람이 표를 보고 판단합니다. §4-6에 적은 판정표가 그 증거입니다. 다음 단계는 플레이 텔레메트리를 게이트로 승격시키는 일입니다.
- 비평자가 사람의 취향을 대리하지 못합니다.** 시각 품질을 판정할 때 비평 에이전트가 세운 지표 다섯을 사람이 통째로 뒤집은 적이 있습니다. 그 뒤로 작업 지시문에 "비평 에이전트를 자동으로 돌리지 마라, 판정 기준은 스크린샷이고 사람이 최종 판정자다"라는 문구를 넣게 됐습니다. 심미 판단을 위임하는 문제는 아직 열려 있습니다.
- 세션 간 기억이 아직 수동입니다.** 실패 패턴을 사람이 손수 메모리 문서로 되먹이고 있는데, 이 루프를 자동화하는 것이 다음 레버입니다.

셋 다 사람이 아직 대신해 주고 있는 판정입니다. 제가 다음에 설계하고 싶은 것도 게임보다는, 이 판정들을 기계에 넘기는 다음 게이트입니다.